Spotify Song Popularity Predictor

Rytham Dawar, Miguel Beltran, Johnny Betmansour, Brian Li, Alexander Yao
UC Davis, Davis, CA, USA, 95616
Email:{rdawar, mmbeltran, jbetmansour, brjli, axyao}@ucdavis.edu

I. Introduction

Music streaming platforms like Spotify have millions of songs to choose from, with hundreds to thousands more uploaded every day. With so much content available, deciding what to recommend or promote becomes an incredibly important and difficult task. This project aims to address this problem by building a machine learning model that predicts a song's popularity.

At the beginning of the project, we focused on predicting popularity using only the audio and musical features of a song. These features describe characteristics of the music itself, such as tempo, loudness, energy, danceability, and other measurable properties. By using only these features, we wanted to explore whether the sound and structure of a song alone could help explain why some songs become more popular than others.

However, after training several models using only audio features, we found that the prediction performance was relatively poor. The models struggled to clearly separate popular songs from less popular ones, which suggested that the audio characteristics alone were not enough to explain popularity. This result makes sense because popularity in music is often influenced by many external factors beyond the sound of the song.

Because of this, we expanded our feature set to include an additional variable called artist_avg_popularity, which represents the average popularity of songs previously released by the same artist. The idea behind this feature is that artists who already have a strong audience or fan base are more likely to release songs that receive attention and streams. By including this information, the model can take into account both the characteristics of the song and the general popularity of the artist.

Using this updated feature set, we trained several machine learning models to predict whether a song would be popular. By including both audio characteristics and the artist's average popularity, the models are able to consider multiple factors that may influence a song's success. This allows the model to capture patterns not only in the sound of the music, but also in the general level of attention an artist tends to receive. Overall, this approach provides a more realistic way to model song popularity using both musical and artist-related information.

II. Literature Review

Aruajo et al. trained machine learning classifiers to predict whether songs appear in Spotify's Top 50 chart based on audio features [1]. A support vector machine (SVM) achieved an accuracy greater than 80% in the testing data, demonstrating that Spotify song features can help distinguish between more popular and less popular tracks. Although this is a classification rather than a regression task, it reinforces the idea that audio features, if appropriately modeled, are sufficient to determine a track's popularity level, validating the concept of our project.

Li directly analyzed Spotify audio features (e.g., danceability, energy, speechiness, duration) and their relationship to song popularity using linear regression [2]. The paper finds that danceability and longer duration are positively associated with higher popularity, as well as that songs in minor keys may be more favored than those in major keys. This work supports the idea that measurable audio features alone can influence and predict popularity scores, even if they cannot account for all external factors. This will help us determine which features we should consider as inputs to our model, and get a general idea of what a linear regression model should look like for this task.

Saragih used a large dataset of Spotify data and compared several machine learning models to identify the best predictors of song popularity [3]. The results highlight that not every audio feature contributes equally. Features such as energy and danceability tend to be consistently predictive, while others are less informative. It also makes use of multiple different machine learning models, and compares their accuracy on both popularity prediction, and popularity classification. This research will be useful in deciding on the features we want to use, as well as deciding on the best model.

III. Dataset Description and Analysis

A. Dataset Description

We are using the Spotify Songs dataset from Kaggle, which contains 30,000 songs collected from the Spotify API. This dataset has 23 features (e.g., danceability, energy, loudness, etc...) which include both numerical and categorical features.

This dataset can be accessed at:

https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs

The dataset's characteristics are:

- Total Songs: 32833
- Total Features: 23
- Target Variable: track_popularity

These 23 features are divided into two different categories:

- Audio Features: danceability, energy, key, loudness, mode, speechiness, acousticness, instrumentalness, liveness, valence, tempo, duration_ms
- Metadata Features: track_id, track_name, track_artist, track_popularity, track_album_id, track_album_name, track_album_release_date, playlist_name, playlist_id, playlist_genre, playlist_subgenre

B. Exploratory Data Analysis

Our dataset contains 32833 rows and 23 columns. It contains 6603 duplicates by (track_name, track_artist) and 4477 duplicates by track_id. The feature we plan to predict, track_popularity, has a mean of 42.477081, standard deviation of 24.984074, minimum of 0, maximum of 100 and skew of -0.23332. It skews heavily to the left with a large cluster of points at 0.

Our exploratory data analysis examines this Spotify dataset to understand the characteristics of our target variable (track_popularity) and its relationship with different audio features (duration_ms, tempo, loudness, danceability, energy, valence, speechiness, acousticness, and instrumentalness). The analysis focuses on three key aspects: data skewness, correlation analysis, and outlier detection.

a) Data Skewness: The distribution of track_popularity (Figure 1, top left) shows a significant left skew, with a large number of songs having very low popularity scores.

Almost 4,000 songs have popularity scores near zero, and the distribution decreases as the popularity increases. The histogram shows that most songs fall in the 30-70 popularity range after the initial spike at zero. This left skew indicates that most songs in the dataset have low popularity, which could require data transformation or other techniques during the development of the model to prevent bias towards predicting low popularity scores.

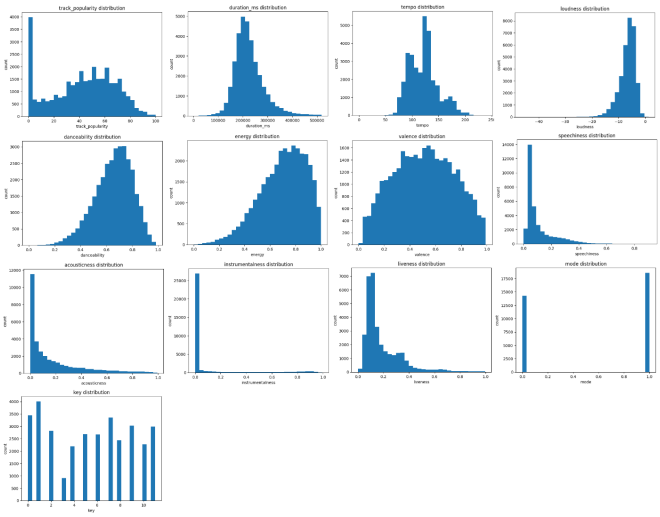

Fig. 1. Distributions of the various audio features. The majority of the features demonstrate some kind of skew, with varying intensities. Very few features have relatively normal distributions, those being valence and tempo.

The distributions of the other audio features show varying patterns of skewness. Speechiness, acousticness, and instrumentalness all show extreme right skewness, with most of the songs concentrated near zero. Duration_ms, danceability, valence, and energy are relatively symmetric and feature more normal distributions, with danceability and energy being slightly left skewed. Loudness features an extreme left skewness, with most of the songs concentrated towards the lower levels. Lastly, tempo has an irregular distribution, with peaks around 100 and 120-130 BPM. Features that have a high degree of skewness may not be strong features to select for our model, or would require data transformation.

b) Correlation Analysis: The correlation heatmap (2) shows relationships between audio features and track popularity, along with correlations between audio features.

It appears that none of the selected audio features show a strong linear relationship with track_popularity: the strongest correlation can be seen with acousticness, with r = 0.09. Danceability and loudness have the second-highest correlation, at r = 0.06. The overall weak correlation between track_popularity and the individual audio features suggests that track_popularity might not be able to be reliably predicted using any single audio

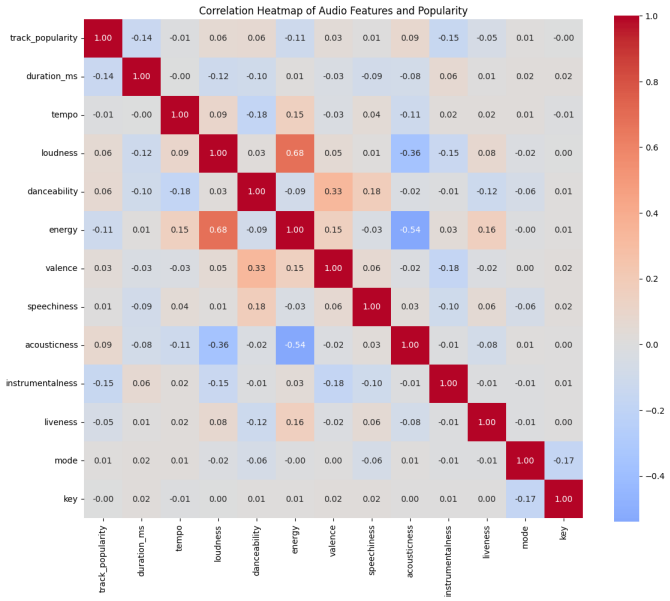

Fig. 2. Correlations of audio features with each other. Most features have weak correlations with each other, particularly with track_popularity. Some of the strong correlations may suggest colinearity.

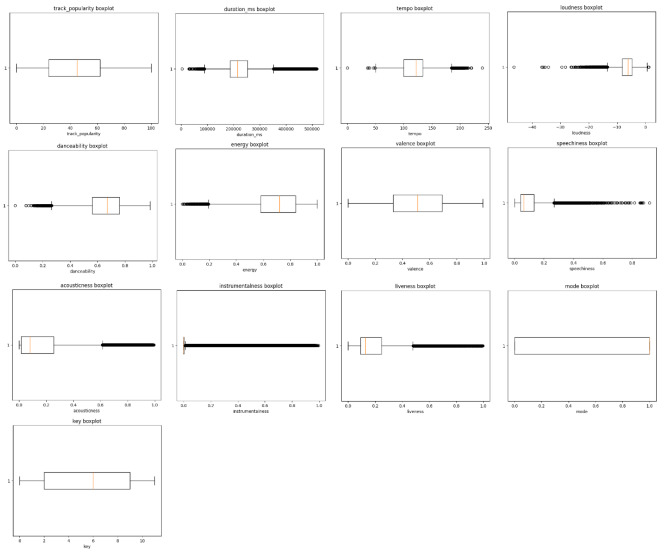

Fig. 3. Boxplots of audio features. Several features contain multiple outliers, particularly on the upper end. Overall, very few features resemble a normal distribution.

feature alone, instead depending on more complex interactions between multiple features.

The heatmap also reveals some strong and weak correlations between predictor variables. For example, loudness and energy have the most positive correlation at r = 0.68, whereas acousticness and energy have the most negative correlation at r = -0.54.

c) Outlier Detection: By plotting box-plots for our target variable and predictor variables (Fig. 3), we were able to identify outliers across various features.

The medians and interquartile range present in the box-plots confirm that our distributions created for data skewness are accurate. Duration_ms is the only feature that shows numerous outliers on both the lower and upper ends. Tempo also displays outliers on both ends, although with significantly more outliers present in the upper end. Loudness is the only feature that contains a significant number of outliers on the lower end, which represents songs that are quieter compared to the typical loudness range. Speechiness, acousticness, and instrumentalness all contain extensive outliers on the upper end, although these outliers may represent subgenres of music, rather than data errors. Lastly, danceability, energy, and valence all show relatively few outliers, with most data points falling within the expected range.

The box-plot for track_popularity (bottom-right) shows no extreme outliers, indicating that the popularity scores are all within the expected range.

IV. Methodology

A. Data Preprocessing

To preprocess our data, we remove duplicate songs, encode categories, split the data into a train and test set, create an artist popularity feature, clip outliers and scale numeric features. These preprocessing steps are important because raw datasets often contain inconsistencies or values that can negatively affect model training. By standardizing the data before modeling, we ensure that the machine learning algorithms can learn meaningful patterns rather than being influenced by noise or formatting differences. After creating a copy of the dataset, we drop duplicate songs to remove repeat song entries to make sure the same song isn't able to appear in both the train and test set. This step helps prevent data leakage, where the model could accidentally learn from the same example during training and evaluation, which would lead to overly optimistic performance estimates. We then one-hot encode the categorical columns, turning categories like playlist genre and playlist subgenre into binary columns. This transformation allows categorical information to be represented in a format that machine learning models can process numerically while still preserving the distinct categories. We also checked whether release year correlated with popularity, but found a negligible correlation (r = 0.06) and excluded it. After one-hot encoding and adding the artist popularity feature,

the final feature set contained 41 features. The dataset is then split into a training set for learning and a test set for evaluation. We use an 80/20 split so that the majority of the data is available for model training while still reserving a separate portion for evaluating model performance on unseen examples.

To help with our model accuracy, we later decided to include a feature providing information about the artist who made the song. One-hot encoding every single artist was unrealistic, as there are approximately 10,000 unique artists in the dataset, and representing each one as its own category would have caused a large increase in dimensionality. This would create thousands of additional columns and make the model more complex without necessarily improving performance. To remedy this, we defined a new feature called artist_avg_popularity and computed it for each artist using their average track popularity.

The artist_avg_popularity is computed by the artist's average track popularity in the training set only. This ensures that the feature is created without using any information from the test set, which prevents information leakage during evaluation. If the artist is absent from the dataset, the overall training artist popularity mean is used instead. This fallback value ensures that songs from artists not present in the training data can still be processed by the model without introducing missing values. Using the overall mean also prevents the model from assigning extreme or undefined values to unseen artists. The feature captures the general level of attention an artist tends to receive, which may influence how likely a new track is to become popular. In this way, the model can incorporate information about artist reputation without needing to represent each artist as a separate category.

As for dealing with outliers, we computed lower and upper IQR bounds using the 25th and 75th percentile, compute IQR = Q3 - Q1 and define the lower bound as Q1 - 1.5 * IQR and upper bound as Q3 + 1.5 * IQR. The interquartile range method provides a simple way to identify unusually large or small values while still keeping the majority of the data unchanged. Anything outside this range is considered an outlier. We then apply outlier clipping using the bounds so that values below the lower bound are set to the lower bound and values above the upper bound are set to the upper bound. Clipping helps reduce the influence of extreme values that could otherwise dominate the learning process, especially for models that are sensitive to large numeric ranges. We chose to apply clipping instead of dropping all outliers to keep the dataset in a reasonable range that is able to generalize well. Removing all outliers could significantly reduce the dataset size and potentially discard useful information, while clipping keeps the observations but limits their effect on the model. Finally, we scale our data so that the numeric columns have a mean of 0 and a standard deviation of 1. Feature scaling ensures that variables measured on different numeric ranges contribute more equally during training. We only applied standard scaling to the numeric features, leaving the one-hot encoded columns unscaled since they are already binary. This is especially important for models such as logistic regression which can be sensitive to the scale of input features.

B. Model Training and Hyperparameter Tuning

After preprocessing the dataset, we trained several machine learning models to learn patterns between the input features and song popularity. Our approach was divided into two stages: an initial regression task to predict the exact popularity score, followed by a classification task that predicted whether a song would be considered a hit.

In the regression stage, the goal was to predict the exact popularity value of a song on a scale from 0 to 100. This allowed us to evaluate whether the audio characteristics of a song alone were enough to explain its popularity. We tested several regression models with different levels of complexity, including Linear Regression, K-Nearest Neighbors Regressor, Random Forest, XGBoost, and a Neural Network. These models were chosen to represent a mix of linear and tree-based learning methods, as well as a neural network model capable of capturing nonlinear relationships.

After observing weak performance in the regression stage, we reformulated the problem as a binary classification task. Instead of predicting the exact popularity score, the model would predict whether a song qualified as a "hit." A song would be labeled as a hit if its popularity score was above a particular threshold. We considered multiple options for this threshold. Initially, we wished to define it as a popularity score over 70. However, this was extremely unbalanced, with a class split of about 90/10. In the opposite direction, we considered a threshold at the 50th percentile, but we felt that this would not be truly representative of the most popular songs. We ultimately settled on a middle ground at the 75th percentile of the training set. This transformation simplified the prediction problem and allowed the models to focus on distinguishing highly popular songs from the rest of the dataset.

For the classification stage, we initially trained five models and selected the three best performing models: Logistic Regression, Random Forest, and XGBoost. These models were selected because they represent different modeling approaches. Logistic Regression provides a simple linear classifier, while Random Forest and XGBoost are ensemble tree-based methods that can capture more complex interactions.

To improve model performance, we applied hyperparameter tuning using grid search combined with five-fold cross-validation. Grid search systematically tests different combinations of hyperparameters for each model and selects the configuration that produces the best performance. For logistic regression, we primarily focused on searching for the optimal regularization strength and penalty method. Random Forest was tuned for optimal estimators, max depth, and minimum samples per leaf. XGBoost was tuned for the number of estimators, learning rate, and max depth. As our dataset had a class imbalance (approximately 3:1 ratio of Not Hit to Hit), we set class_weight="balanced" for Logistic Regression and Random Forest, and scale_pos_weight = 2.98 for XGBoost, so that misclassifications on the minority class are penalized proportionally. We also used F1 as our evaluation metric, since we're most interested in whether our models can accurately make decisions on popular songs.

The best hyperparameters found through grid search were: Logistic Regression with C = 10 and L1 penalty, Random Forest with max_depth = 20, min_samples_leaf = 2, and n_estimators = 300, and XGBoost with learning_rate = 0.05, max_depth = 8, and n_estimators = 200. After the optimal hyperparameters were found for all three models, they were trained on the full training dataset and evaluated on the test set. This evaluation allowed us to measure how well each model generalized to unseen data and compare their effectiveness in predicting song popularity.

V. Results and Evaluation

A. Regression

In the regression phase, we planned to predict the exact popularity score using only audio features. Since we were predicting a numerical value, we used Mean Squared Error (MSE) as a metric to see how well the models performed, and the Coefficient of Determination (R^2) to see how well the models explained the data. We tested five distinct models, ranging from simple baselines to neural networks.

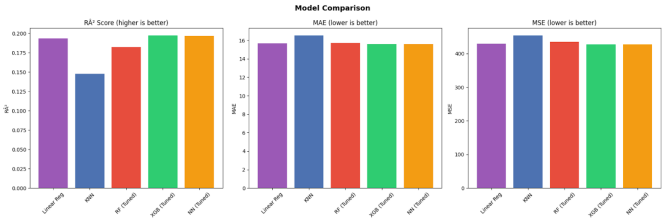

Fig. 4. Comparison of linear regression model R^2, MAE, and MSE scores. All models had a similarly poor performance, with KNN being exceptionally bad.

The first model we tested was Linear Regression; this was an important step as it confirmed our understanding that the audio feature data has a non-linear relationship, as we found during the Exploratory Data Analysis. The performance of this model was poor, with an MSE of 429 and an R^2 score of 19.3%. We also tried another baseline model, K-Nearest Neighbors Regressor; this had an even higher MSE of $\tilde{4}53$ and an R^2 of 14.8%. This was our worst-performing model in both regards (MSE and R^2).

The other models included a Neural Network, which had an MSE of 427 and an R^2 of 19.6%, and a Random Forest model that had an MSE of 435 and an R^2 of 18.2%. As for our best regression model, this was XGBoost, with an MSE of 427 and an R^2 of 19.7%.

These results are not ideal, as the regression models showed very weak predictive performance. The low R^2 ($\tilde{2}0\%$ or lower across all models) shows that 80% of the variance in song popularity is unexplained by the audio features. The MSE was also very high; being roughly 20 to 22 points off from the actual score on a 0–100 range is significant, highlighting that the models had a difficult time predicting the popularity score.

These results from the regression models helped us conclude one thing: audio features (energy, tempo, etc...) do not provide a full picture of track popularity. They only provide the technical profile and do not account for external factors such as social media trends, marketing budgets, or artist popularity, to name a few.

Because these regression models performed poorly in predicting exact scores, we adjusted our problem to a classification approach instead.

B. Classification

a) Experimental Setup: To evaluate whether a song qualifies as a "hit", we framed the task as a binary classification problem. Songs with a popularity score at or above the 75th percentile of the training set (¿=58) were labeled as a "hit" and the remaining songs were labeled

as “not hit”. After removing duplicates and applying the 80/20 train-test split, the training set had 20984 samples (74.9% Not Hit, 25.1% Hit) and the test set contained 5246 samples (75.1% Not Hit, 24.9% Hit). We initially trained five classifiers - KNN, Neural Network, Logistic Regression, Random Forest, and XGBoost. We dropped KNN and Neural Network, which had the lowest baseline F1 scores for simplicity. The remaining three models were tuned using five-fold cross-validated grid search optimizing for F1 score. Then, we would report accuracy, precision, recall, and the F1 score on the test set.

b) Model Comparison: As shown in Figure 5, the three tuned classifiers show distinct trade-offs across the four evaluation metrics.

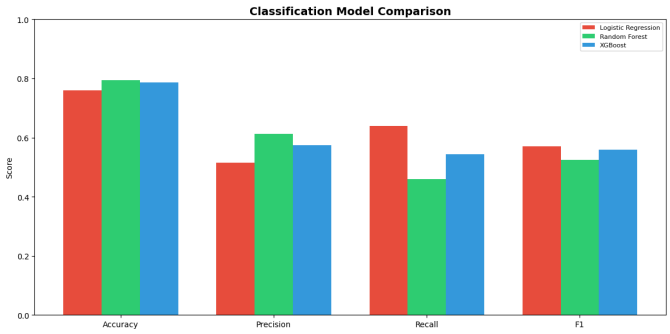

Fig. 5. Comparison of classification model performance metrics. Random Forest demonstrates the highest accuracy, Logistic Regression the highest recall and F1, and XGBoost a middle ground between the two.

Random Forest scored the highest accuracy (0.793) and precision (0.613), meaning that when it predicted a song as a hit, it was correct more often than the other models. However, this came at the cost of recall (0.459), indicating that it missed over half of the actual hits. Logistic Regression achieved the highest recall (0.639) and F1 score (0.570), detecting a larger portion of true hits but creating more false positives, which lowered its precision to (0.514) and its accuracy to (0.760). XGBoost offered a middle ground with an accuracy of (0.786), precision to (0.575), recall of (0.544), and F1 of (0.559). These results suggest that the choice of best model depends on the intended application, if minimizing missed hits is the priority, then Logistic Regression is the choice. If prediction confidence matters more, then Random Forest is the stronger choice. XGBoost offered the best balance across all four metrics. None of its scores were the highest individually, but it avoided the sharp trade-offs seen in the other two models.

c) ROC Curves: Figure 6 shows the Receiver Operating Characteristic (ROC) curves for the three models, plotting the true positive rate against the false positive rate at various classification thresholds.

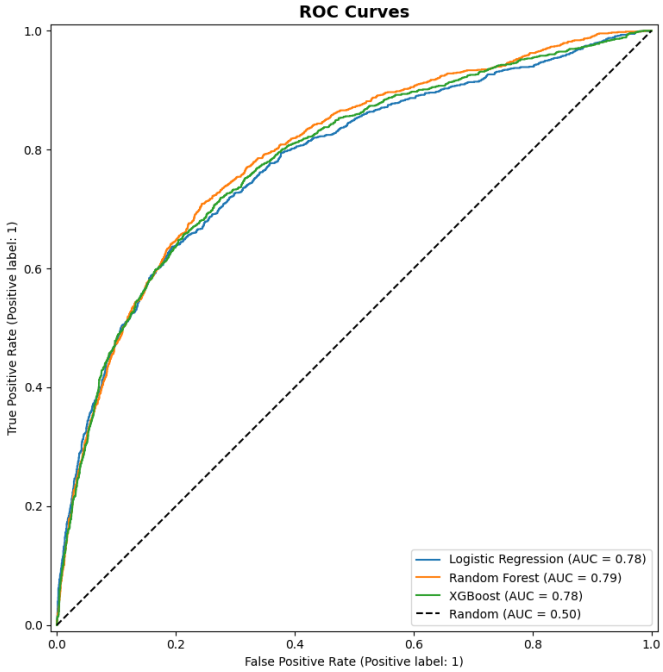

Fig. 6. Classification model ROC curves. All models performed better than the baseline, and performed relatively similarly, suggesting that they all have similar predictive abilities

Random Forest achieved the highest area under the curve (AUC = 0.793), followed by XGBoost (AUC = 0.785) and Logistic Regression (AUC = 0.779). All three models clearly outperformed the random baseline of (AUC = 0.50), which confirms the meaningful discriminative patterns being found. The relatively similar AUC values indicate that the models have similar overall ranking abilities. The differences seen in precision, recall, and F1 largely reflect how each model’s default thresholds interact with the class imbalance.

d) Feature Importance: Figure 7 displays the feature importances from the tuned XGBoost model, with one-hot encoded genre and subgenre columns averaged into single bars so the other features are seen.

The most important feature by a significant margin was artist_avg_popularity (importance = 0.203), an engineered feature which represented the average popularity of each artist’s songs in the training set. This result is expected as well-known artists are far more likely to create hits over lesser known artists, and this engineered feature captures artist-level brand recognition that raw audio features cannot. The next tier of feature importances included playlist genre (0.028), instrumentalness (0.022), and the playlist subgenre (0.021), suggesting that the

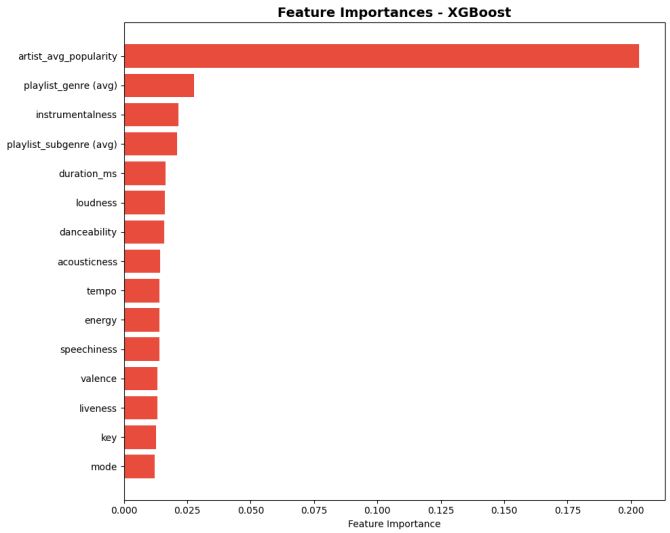

Fig. 7. XGBoost Feature Importances. The most important feature by far being artist_avg_popularity, representing how well-known a given artist is. Other features have relatively equal and low importances.

genre context in which a song appears has a meaningful signal. Among the remaining audio features, duration, loudness, and danceability contributed modestly (around 0.014-0.016), while mode and key were the least informative. We can clearly see that the most important audio feature would be instrumentalness, which can capture a lot of the song's personality as opposed to some of the other more subtle features. On the other hand, the dominance of artist_avg_popularity indicated that song popularity is more driven by who performs the track rather than by its acoustic properties alone.

VI. Conclusion

This project investigated how feasible it was to predict Spotify track popularity using machine learning models trained on audio features, genre categories, and average artist track popularity. Using K-Nearest Neighbors, Random Forest, XGBoost, Neural Networks, and Logistic Regression, we created models across both regression and classification frameworks. From our findings, we have learned more about the predictability of music popularity and the relative importance of different features.

In the regression task, we attempted to use our models to predict the exact popularity scores of a song, and our best-performing model (XGBoost) achieved an R^2 score of only 0.197, explaining less than 20% of the variance in track popularity. This poor performance was consistent across all models, suggesting that audio characteristics, even when supplemented with genre category and artist information, is insufficient for accurately predicting popularity scores. After recognizing this issue, we switched the problem to binary classification, distinguishing between "Hit" songs (top 25%) and "Not Hit" songs (bottom 75%). By switching to this approach, we were able to see substantially better results, with Logistic Regression, Random Forest, and XGBoost achieving F1 scores of around 0.525-0.57.

The big performance gap between regression and classification reflects the fact that the specific factors driving popularity are largely vague. There are possibly numerous factors affecting song popularity that were not included in our dataset, including marketing campaigns, social media trends, artist-fan relationships, and more. Our feature importance analysis revealed that artist average popularity, song genre/subgenre, and instrumentalness were the most influential factors in determining a song's popularity, with our engineered artist_avg_popularity feature dominating the other features. This fact reinforces the idea that artist fame plays a critical role in determining track popularity, which is a factor that extends beyond the audio characteristics of the music itself. As our engineered feature was extremely successful, it also demonstrates that good feature engineering based around knowledge of the subject matter can substantially improve model performance, even when working with relatively limited data.

Additionally, our models lacked access to external variables such as marketing budgets, social media engagement, and radio play. These are factors that may be more predictive of track popularity than audio features. Furthermore, the popularity scores in our dataset represent accumulated popularity over time, meaning that they include effects from post-release promotion and other events that happened after the song was released. Again, our models did not have access to this information when making predictions. In the future, our work could be improved upon by including social media data, marketing information, and release timing data, or by using deep learning models that can analyze the actual audio waves instead of relying on pre-calculated audio features.

In conclusion, while our models demonstrate that machine learning can provide moderately accurate predictions for identifying potential hit songs, the majority of the variance in music popularity remains unexplained when using audio and genre features alone. This demonstrates that musical success in the modern age is complex and largely dependent on external factors mainly relating to society, something that extends far beyond the acoustic properties present in our dataset.

VII. Roadmap and Milestones

1) Weeks 1-2: Dataset Selection, Brainstorming, Problem Statement, One-page Project Proposal
2) Weeks 3-4: Exploratory Data Analysis, Feature Selection and Analysis, Data Preprocessing, Mid-Quarter Project Report
3) Weeks 5-6: Regression Models, Classification Models, Hyperparameter Tuning, Evaluation, Front-end Design
4) Week 7: Final Report, Presentation Slides and Preparation

VIII. GitHub

The GitHub repository with our Juypter Notebook, dataset, and frontend can be found at: https://github.com/Rytham1/song-popularity-predictor/

References

[1] C. V. Soares Araujo, M. A. Pinheiro de Cristo, and R. Giusti, "A Model for Predicting Music Popularity on Streaming Platforms," Revista de Informática Teórica e Aplicada, vol. 27, no. 4, pp. 108–117, Dec. 2020, doi: https://doi.org/10.22456/2175-2745.107021.

[2] K. Li, "Predicting song popularity in the digital age through Spotifys data," Theoretical and Natural Science, vol. 39, no. 1, pp. 68–75, Jun. 2024, doi: https://doi.org/10.54254/2753-8818/39/20240600.

[3] Harriman Samuel Saragih, "Predicting song popularity based on spotify's audio features: insights from the Indonesian streaming users," Journal of Management Analytics, vol. 10, no. 4, pp. 693–709, Jul. 2023, doi: https://doi.org/10.1080/23270012.2023.2239824.

[4] F. Pachet and P. Roy, "Hit Song Science Is Not Yet a Science," in *Proc. 9th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Philadelphia, PA, USA, 2008, pp. 355–360. [Online]. Available: https://www.researchgate.net/publication/220723429

[5] M. J. Salganik, P. S. Dodds, and D. J. Watts, "Experimental Study of Inequality and Unpredictability in an Artificial Cultural Market," *Science*, vol. 311, no. 5762, pp. 854–856, Feb. 2006, doi: https://doi.org/10.1126/science.1121066.